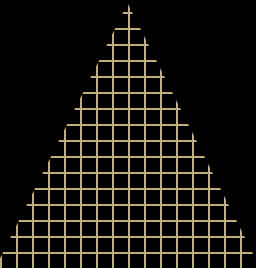


CS-218 Data Structures and Algorithms

LECTURE 19

BY UROOJ AINUDDIN





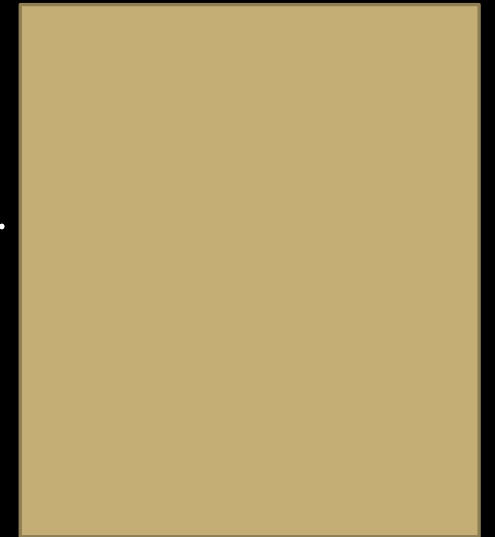
Trees

BOOK 1 CHAPTER 13

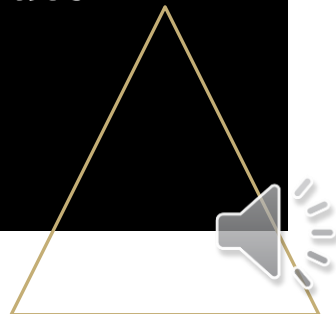


The Tree Structure

- A tree structure consists of **nodes** and **edges** that organize data in a hierarchical fashion.
- The data elements are stored in nodes and pairs of nodes are connected by edges.
- The directed edges represent the relationship between the nodes that they link together.
- The edges are directed downwards.
- Formally, we can define a tree as a set of **nodes** that either is empty or has a node called the **root** that is connected by edges to zero or more subtrees to form a hierarchical structure.

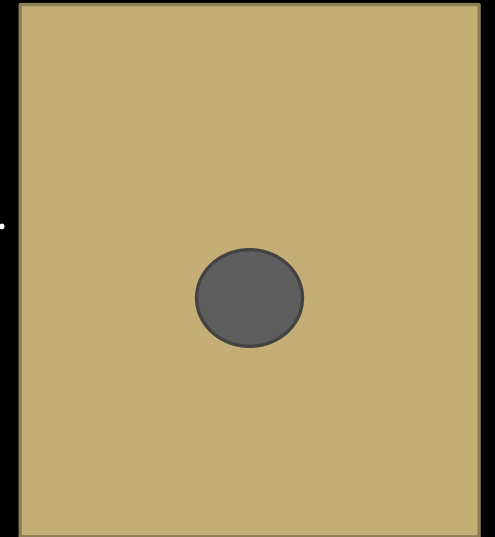


Tree with 0
nodes

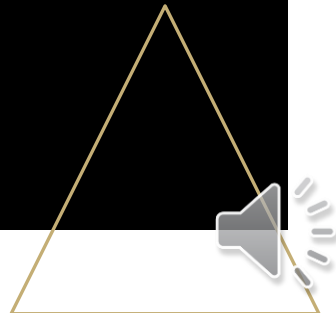


The Tree Structure

- A tree structure consists of **nodes** and **edges** that organize data in a hierarchical fashion.
- The data elements are stored in nodes and pairs of nodes are connected by edges.
- The directed edges represent the relationship between the nodes that they link together.
- The edges are directed downwards.
- Formally, we can define a tree as a set of **nodes** that either is empty or has a node called the **root** that is connected by edges to zero or more subtrees to form a hierarchical structure.

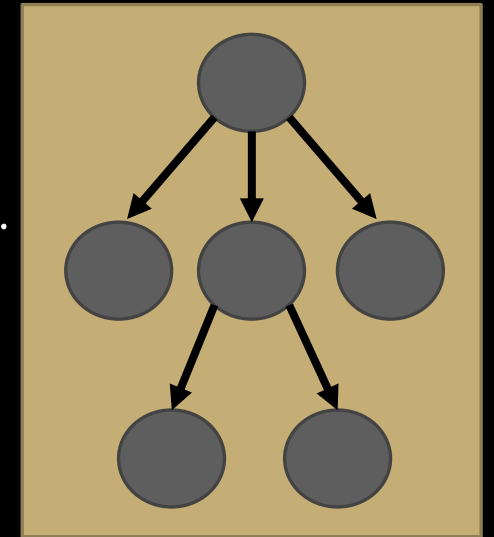


Tree with 1
node



The Tree Structure

- A tree structure consists of **nodes** and **edges** that organize data in a hierarchical fashion.
- The data elements are stored in nodes and pairs of nodes are connected by edges.
- The directed edges represent the relationship between the nodes that they link together.
- The edges are directed downwards.
- Formally, we can define a tree as a set of nodes that either is empty or has a node called the root that is connected by edges to zero or more subtrees to form a hierarchical structure.
- Each subtree is itself a tree.



Tree with 6
nodes



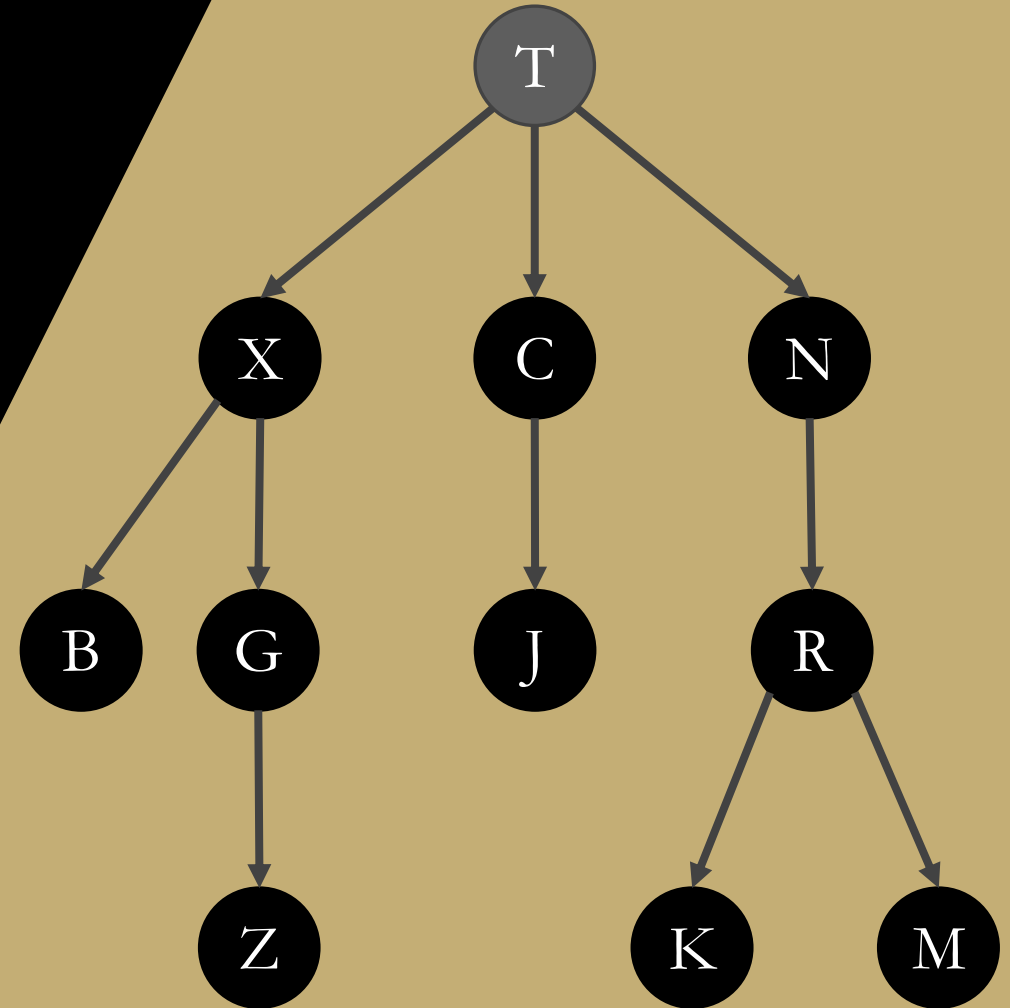


Examples of trees

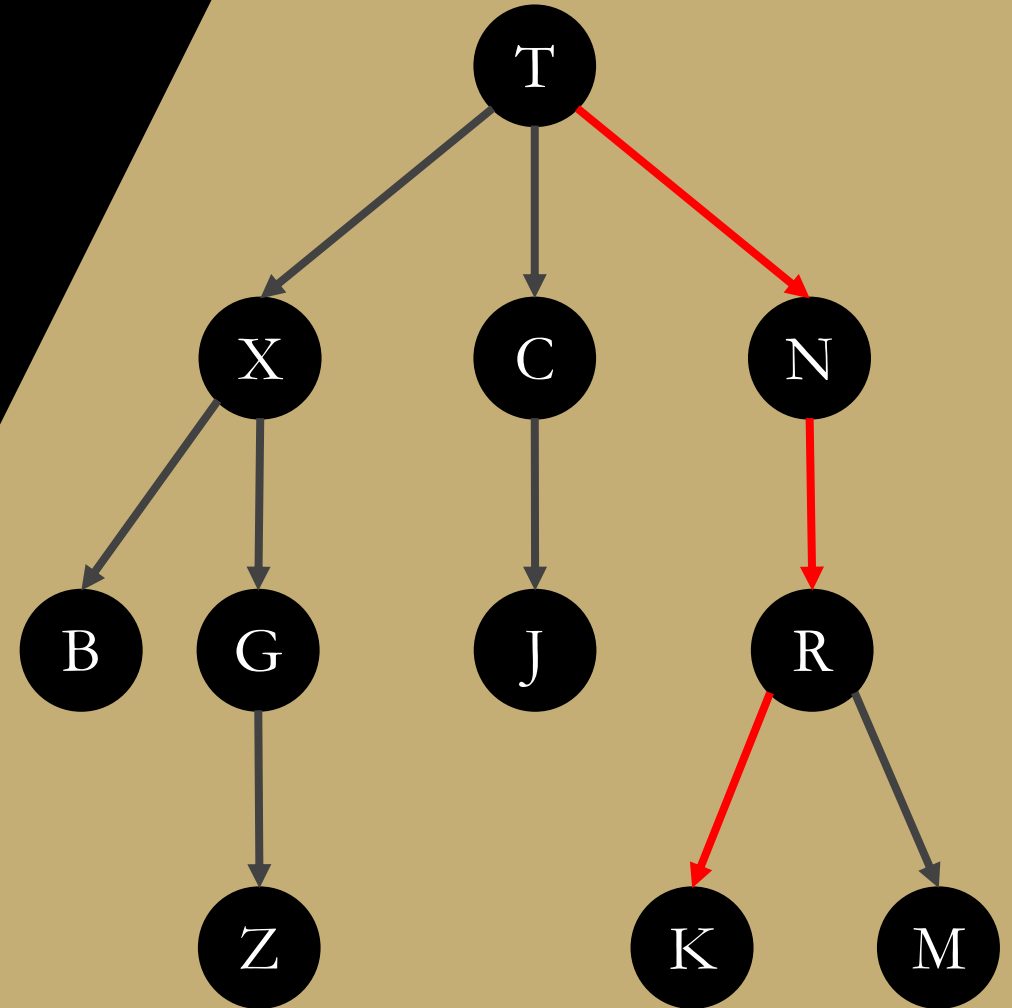
- Family trees are tree structures that depict the relationships between different generations in a group of people related by blood or marriage.
- A classic example of a tree structure is the representation of directories and subdirectories in a file system.



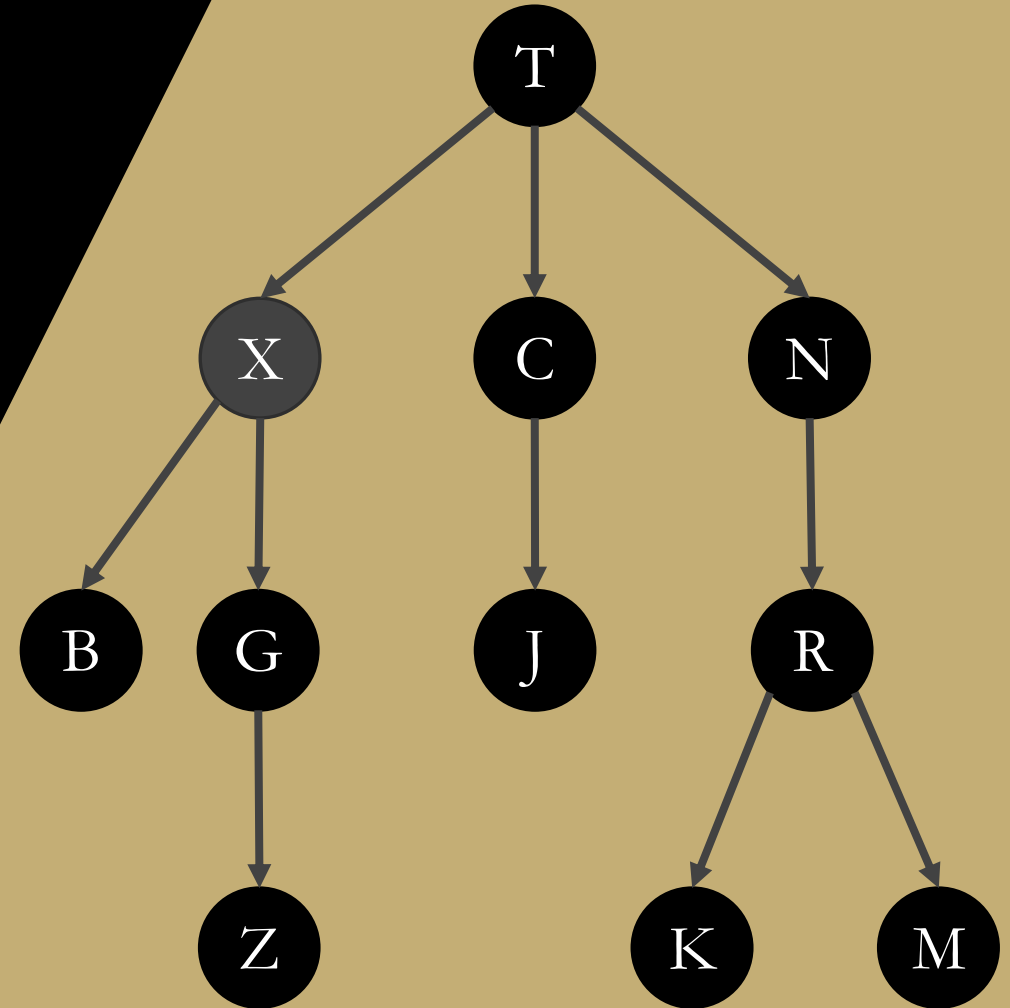
- The topmost node of the tree is known as the **root** node.
- It provides the single access point into the structure.
- The root node is the only node in the tree that does not have an incoming edge (an edge directed toward it).
- The root of this tree is T.



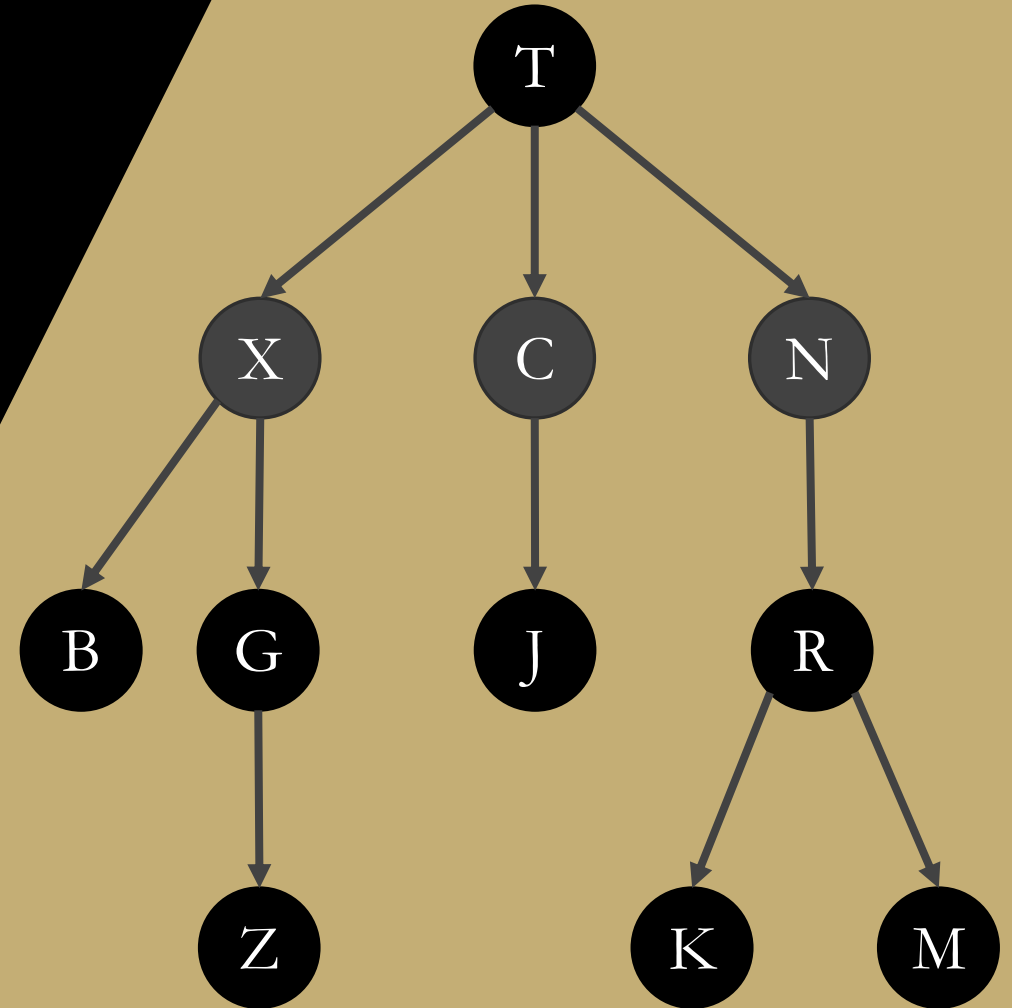
- The other nodes in the tree are accessed by following the edges from a starting node to a destination. The nodes encountered form a **path**.
- The length of a path is the number of edges that form it.
- The sequence $\langle T, N, R, K \rangle$ forms a path from T to K of length 3.



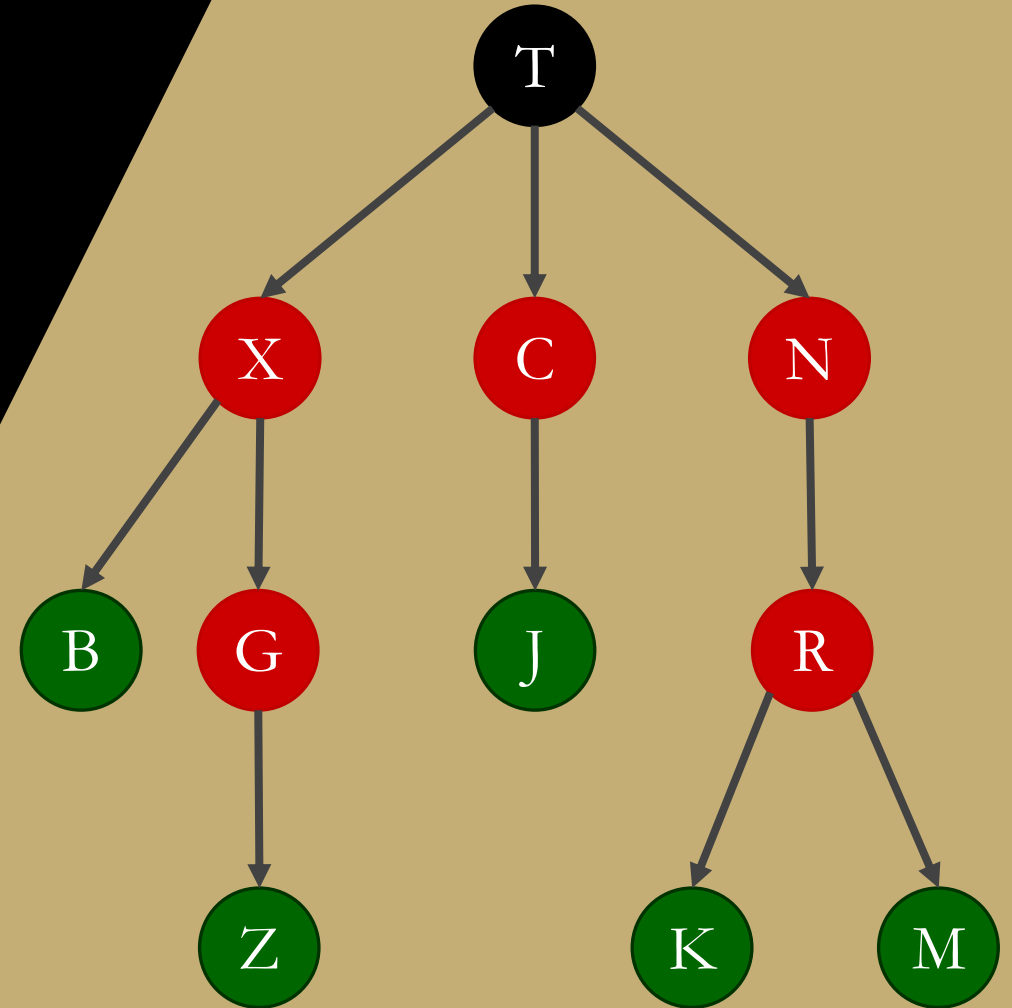
- Every node, except the root, has a **parent** node, which is identified by the incoming edge.
- A node can have only one parent (or incoming edge) resulting in a unique path from the root to any other node in the tree.
- X is the parent of both B and G.



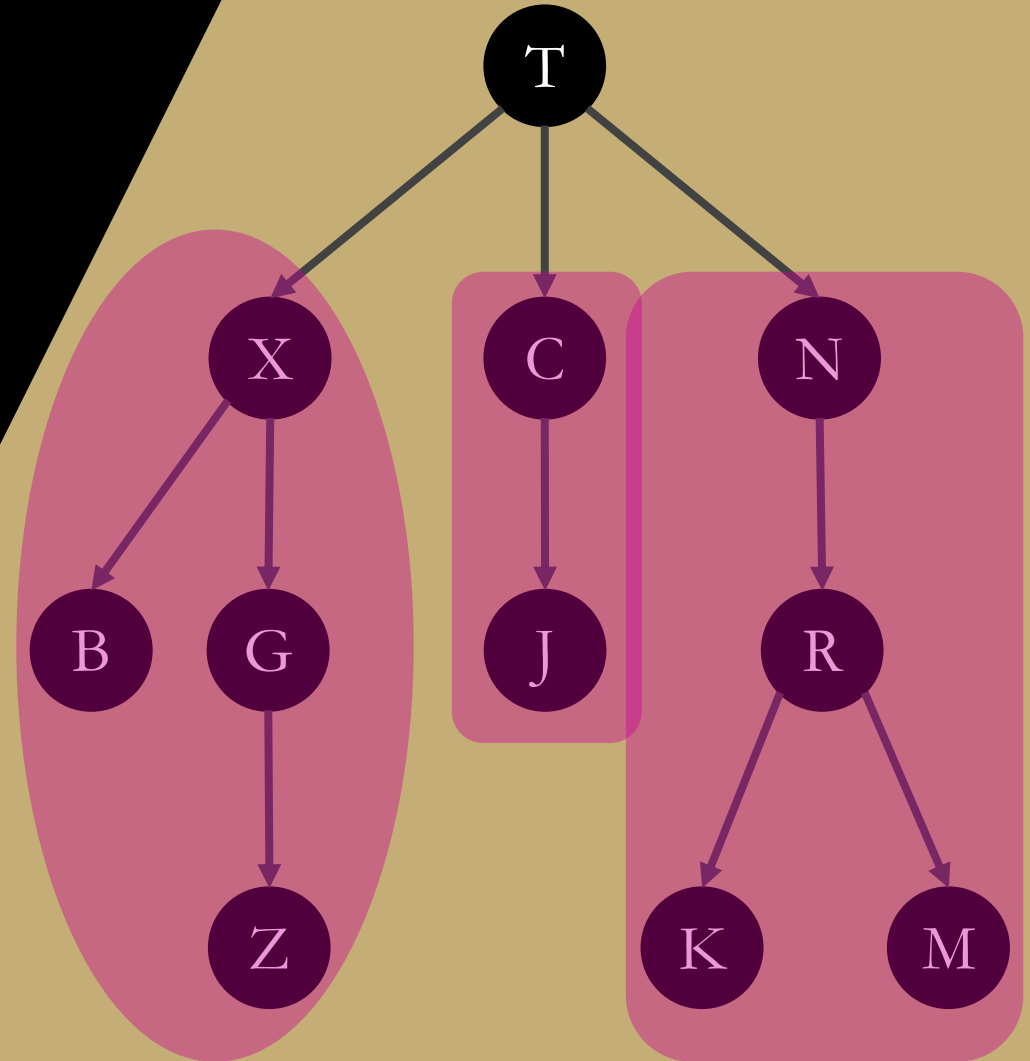
- Each node can have one or more **child** nodes.
- The children of a node are identified by the outgoing edges (directed away from the node).
- All nodes that have the same parent are known as **siblings**, but there is no direct access between siblings.
- T has 3 children: X, C and N.



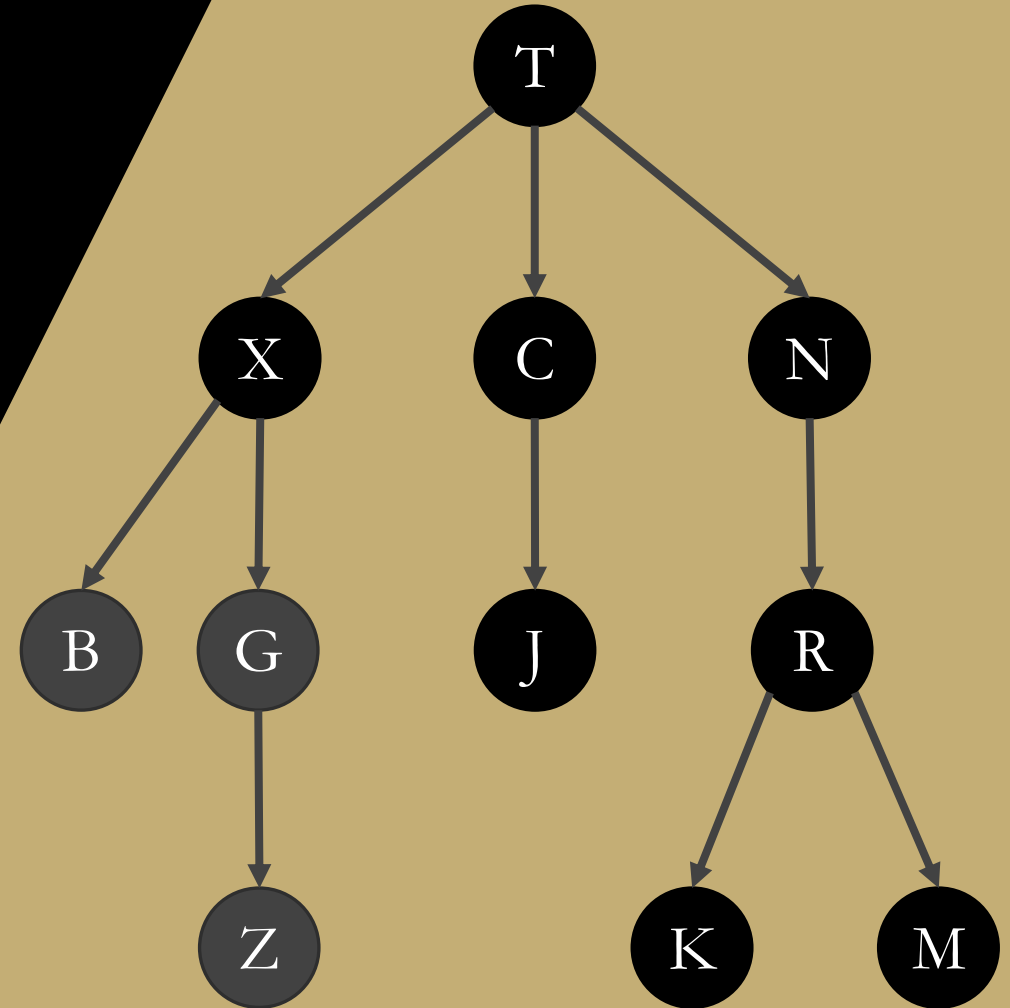
- Nodes that have at least one child are known as **interior nodes**.
- Nodes that have no children are known as **leaf nodes**.
- The interior nodes are: X, C, N, G and R.
- The leaves are: B, J, Z, K and M



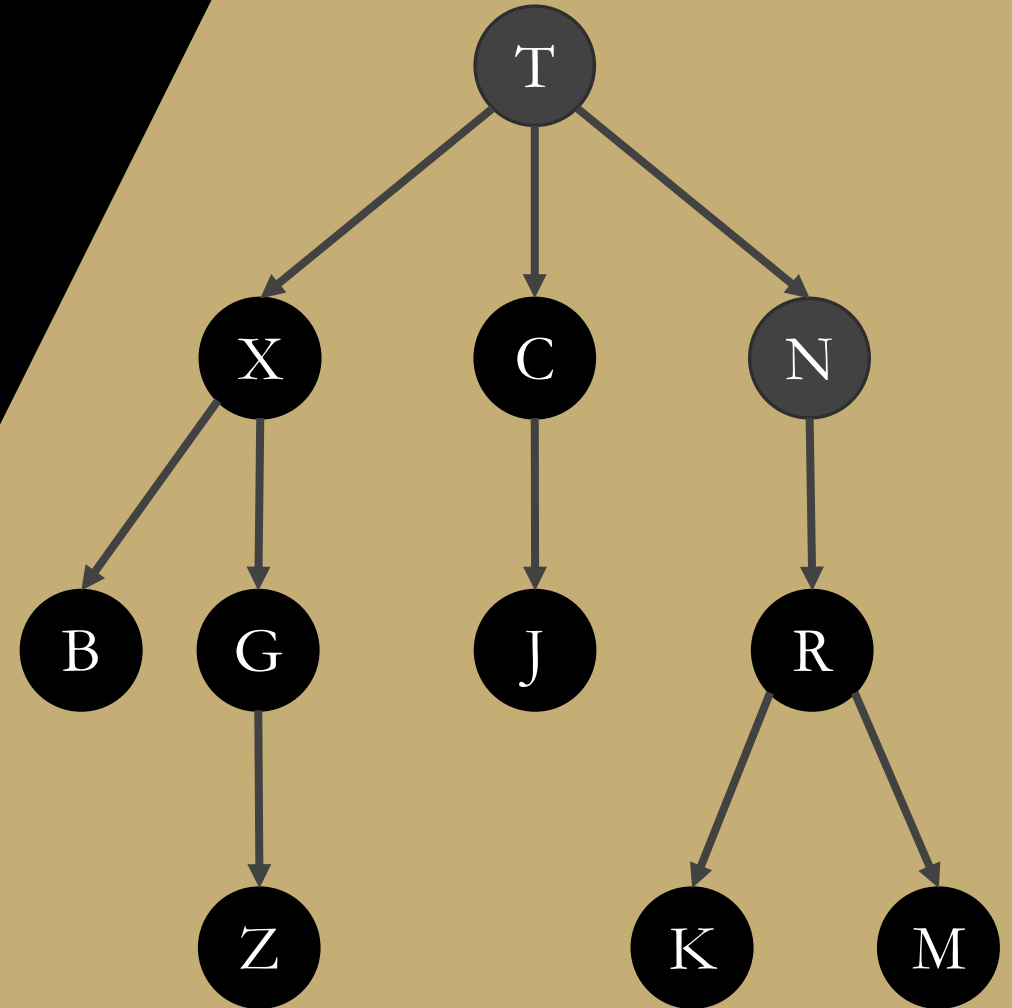
- A tree is a **recursive** structure.
- Every node can be the root of its own subtree, which consists of a subset of nodes and edges of the larger tree.
- T has three subtrees rooted at X, C and N.



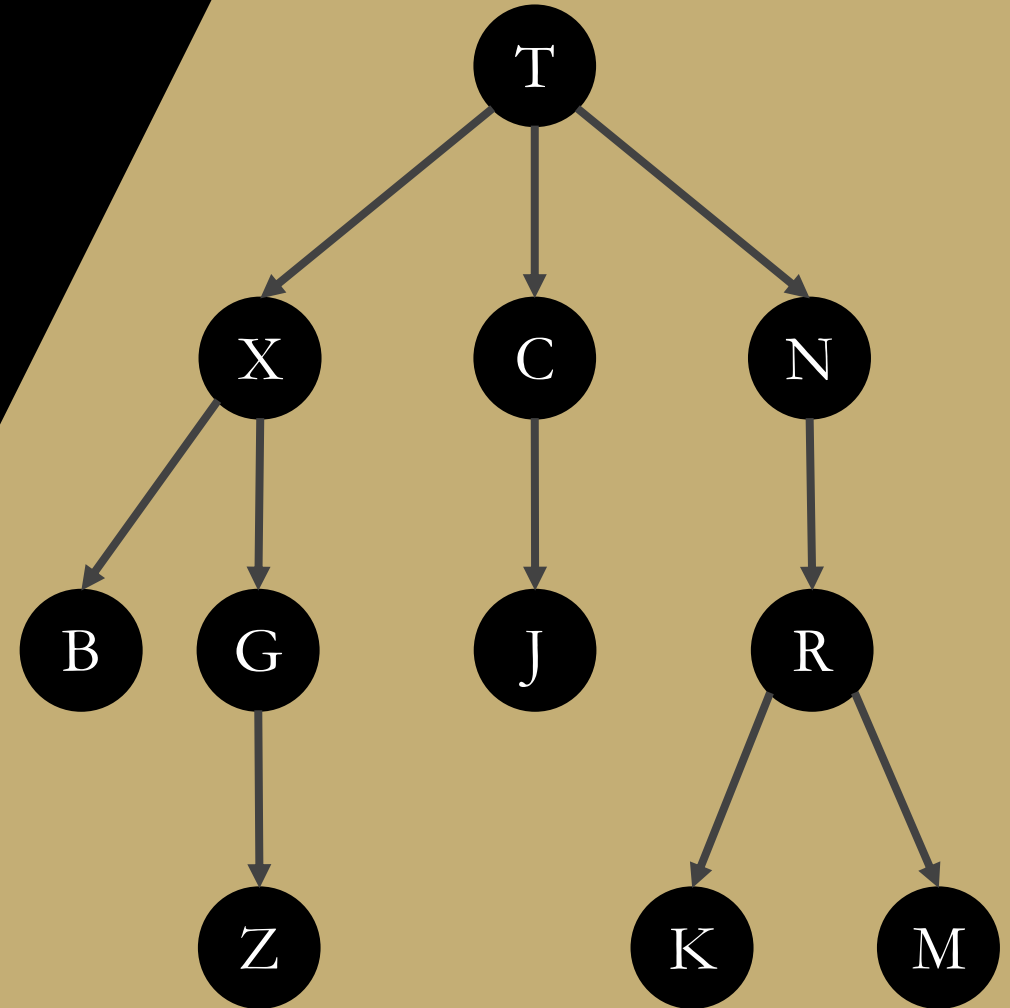
- The **descendants** of a node are its children and their children all the way down to (and including) the leaves.
- Every node in the tree is a descendant of the root node.
- The descendants of X are B, G and Z.



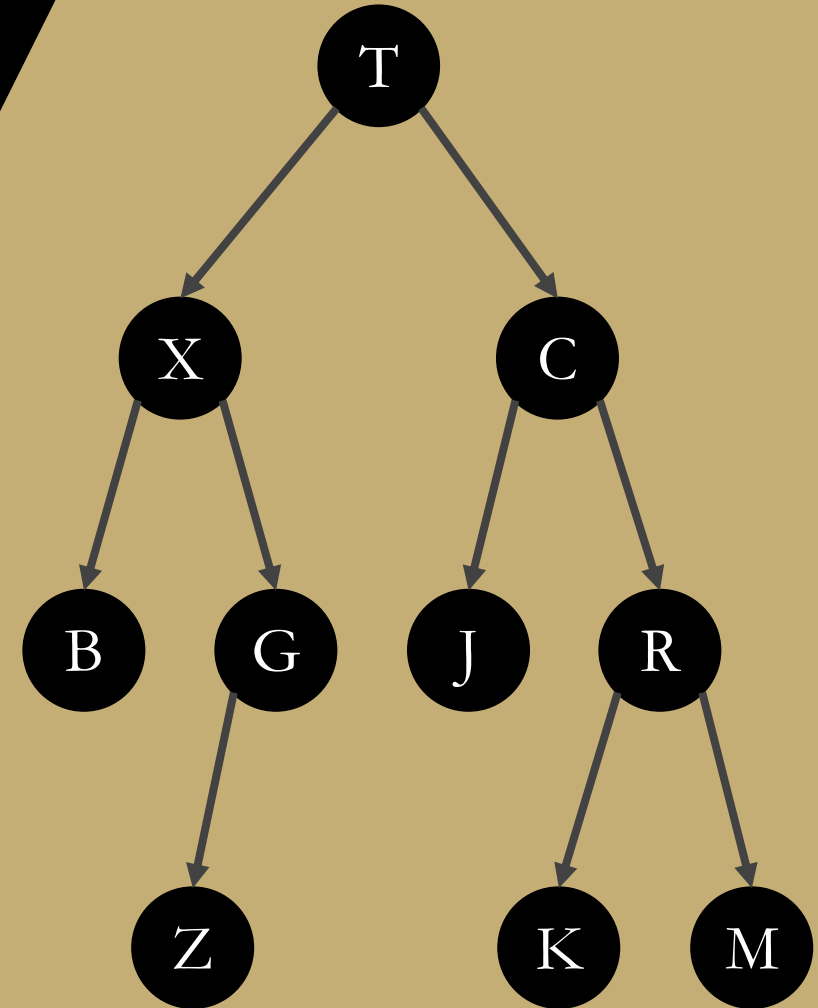
- The **ancestors** of a node are the nodes along the path from the root to itself.
- The root node is the ancestor of every node in the tree.
- The ancestors of R are T and N.



- One of the most used trees in computer science is the **binary tree**.
- A binary tree is a tree in which each node can have at most two children.
- One child is identified as the **left child** and the other as the **right child**.
- In the remainder of this discussion, we focus on the use and construction of the binary tree.
- This is not a binary tree.



- One of the most used trees in computer science is the **binary tree**.
- A binary tree is a tree in which each node can have at most two children.
- One child is identified as the **left child** and the other as the **right child**.
- In the remainder of this discussion, we focus on the use and construction of the binary tree.
- This is a binary tree.



Binary trees

- The nodes in a binary tree are organized into levels.
- The root node is at level 0, with its children at level 1.
- The children of level 1 nodes are at level 2, and so on.

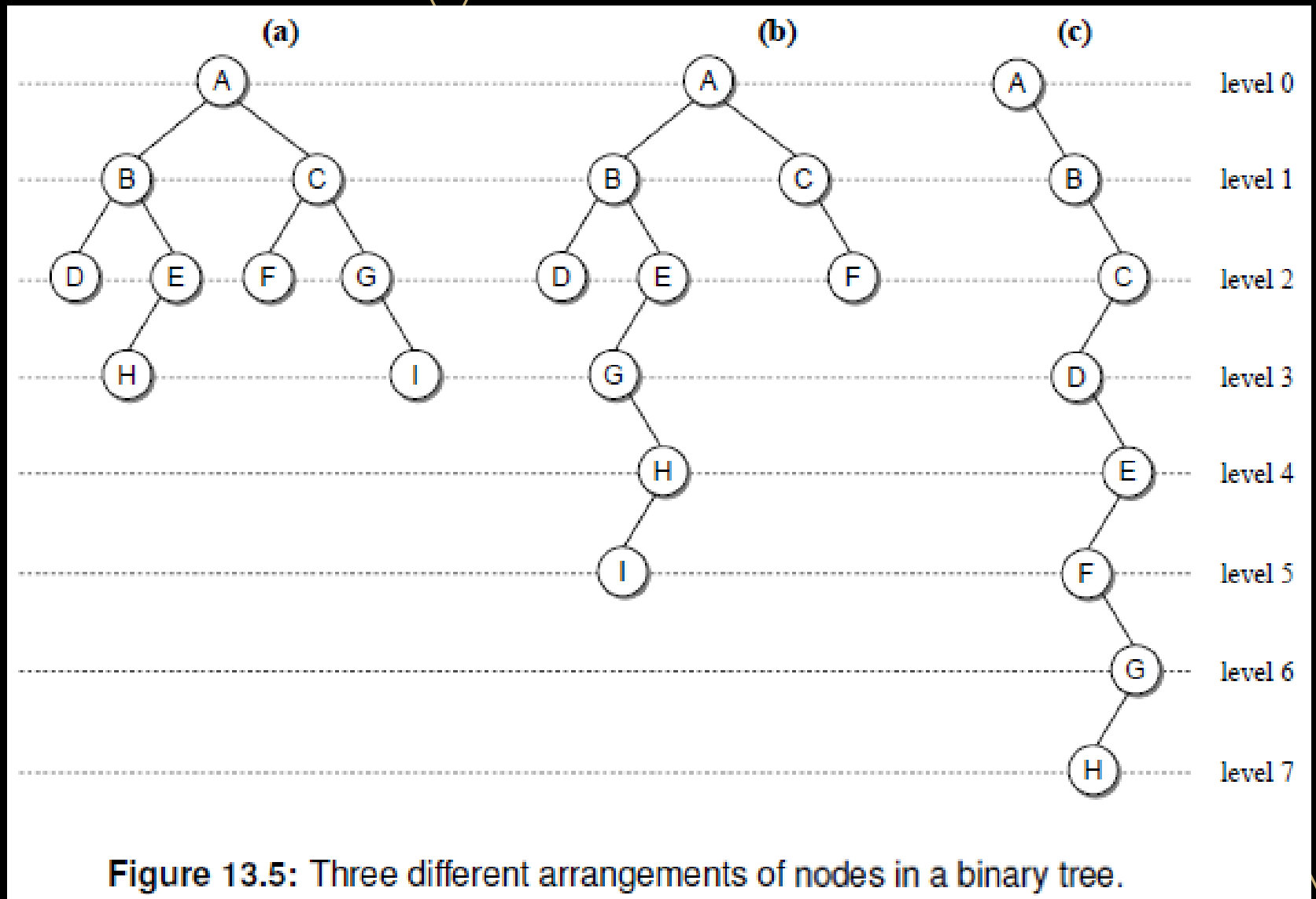


Figure 13.5: Three different arrangements of nodes in a binary tree.

Binary trees

- The **depth** of a node is its distance from the root.
- A node's depth corresponds to the level it occupies.

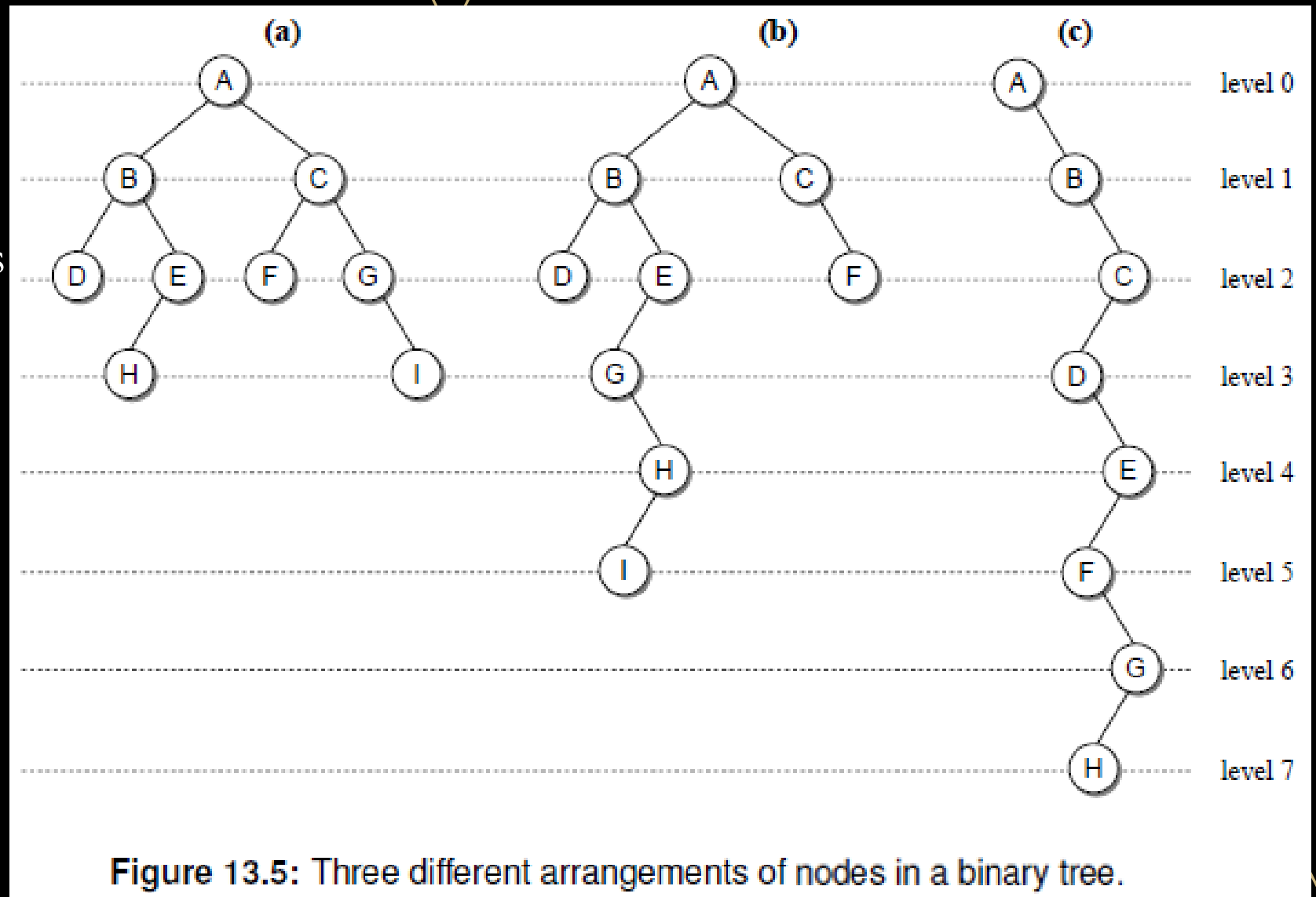
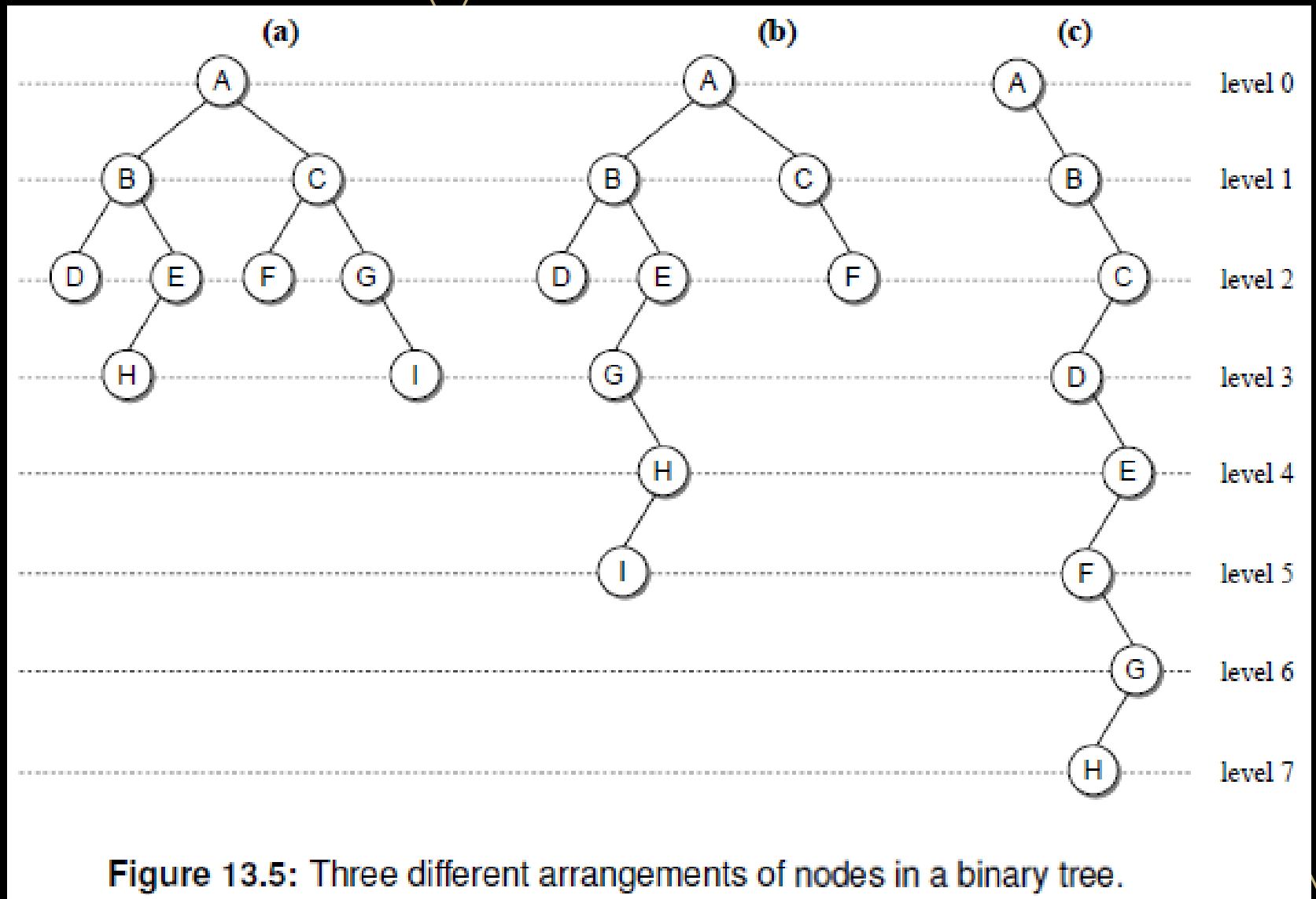


Figure 13.5: Three different arrangements of nodes in a binary tree.

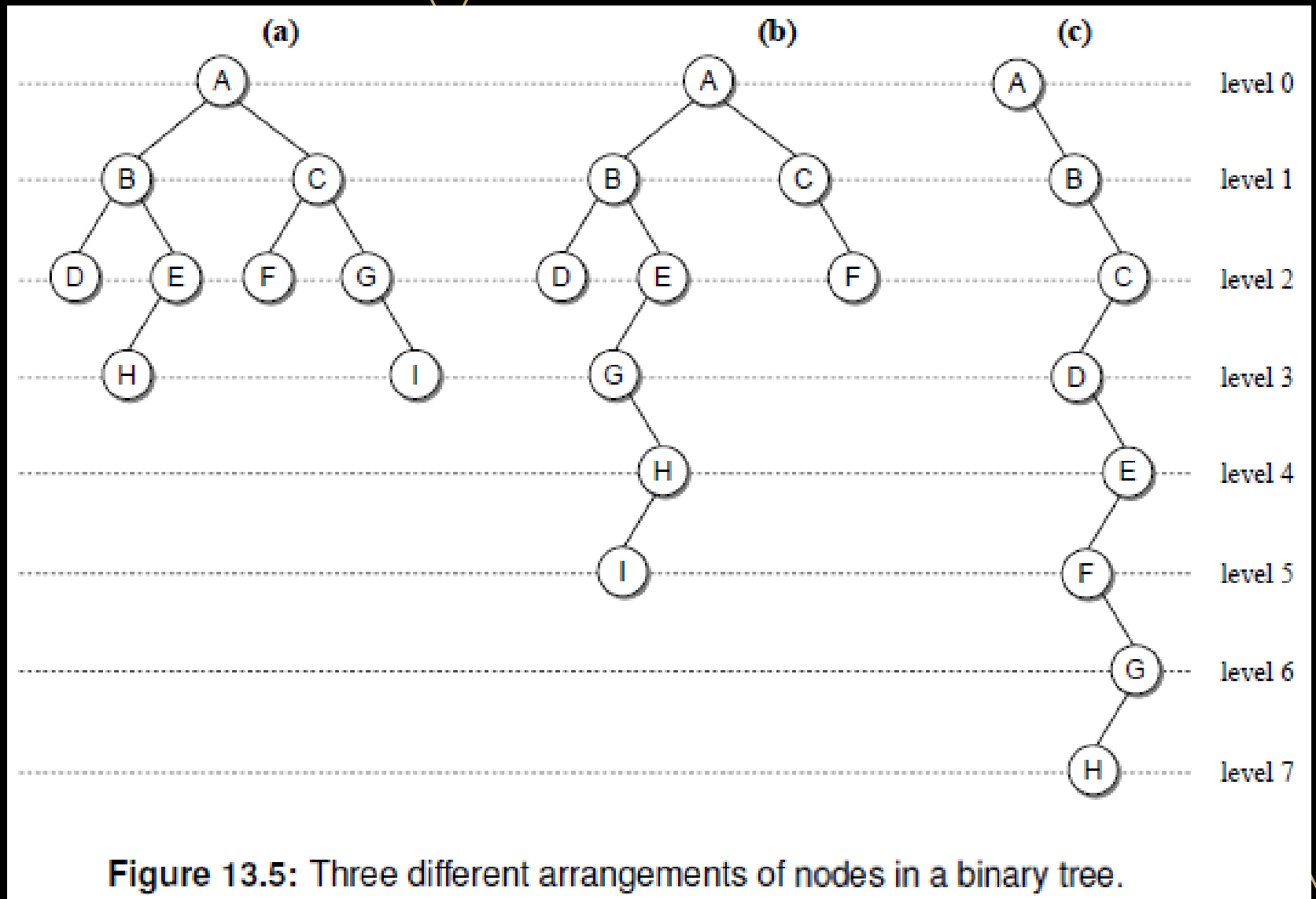
Binary trees

- The **height** of a binary tree is the number of levels in the tree.
- The last level in a tree of height h is $h - 1$.
- The height of tree (a) is 4.
- The height of tree (b) is 6.
- The height of tree (c) is 8.



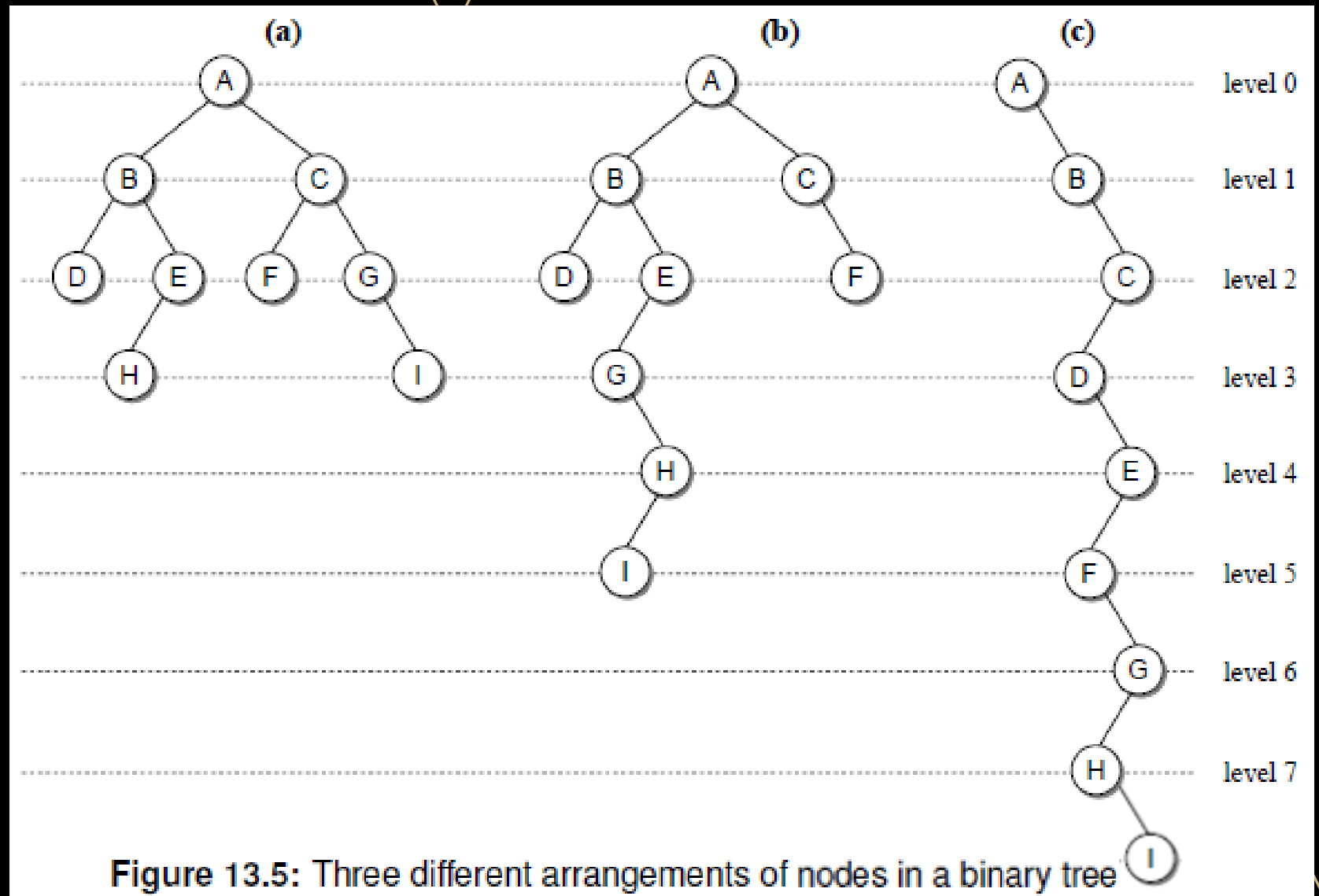
Binary trees

- The **width** of a binary tree is the number of nodes on the level containing the most nodes.
- The width of tree (a) is 4.
- The width of tree (b) is 3.
- The width of tree (c) is 1.



Binary trees

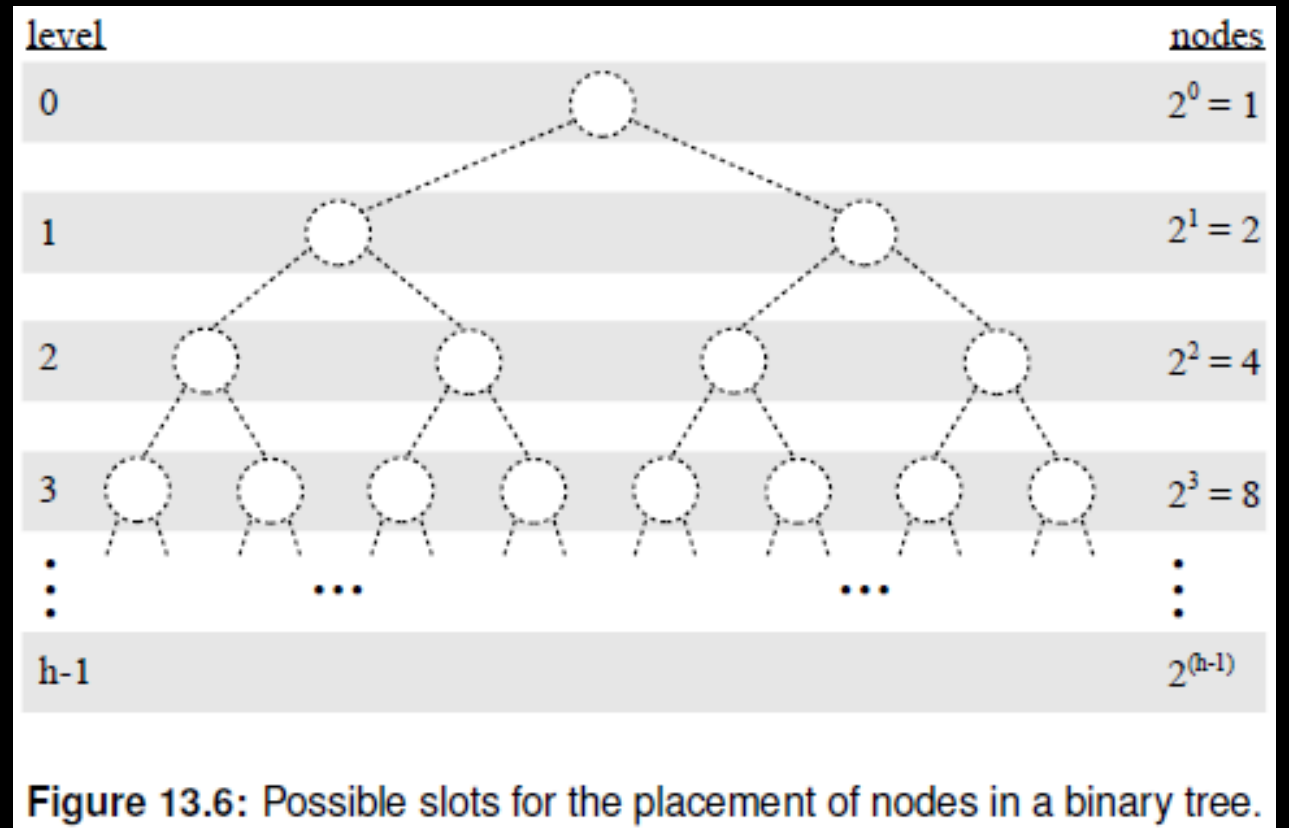
- The **size** of a binary tree is the number of nodes in the tree.
- The size of all three trees is 9.
- A binary tree of size n has a **maximum** height of n , when there is one node per level.



Binary trees

- A binary tree of n nodes will have the **minimum** height when all possible slots in it contain nodes, and no slot is empty.

Level#	Nodes
0	2^0
1	2^1
2	2^2
$\vdots$	$\vdots$
$h - 1$	2^{h-1}



Binary trees

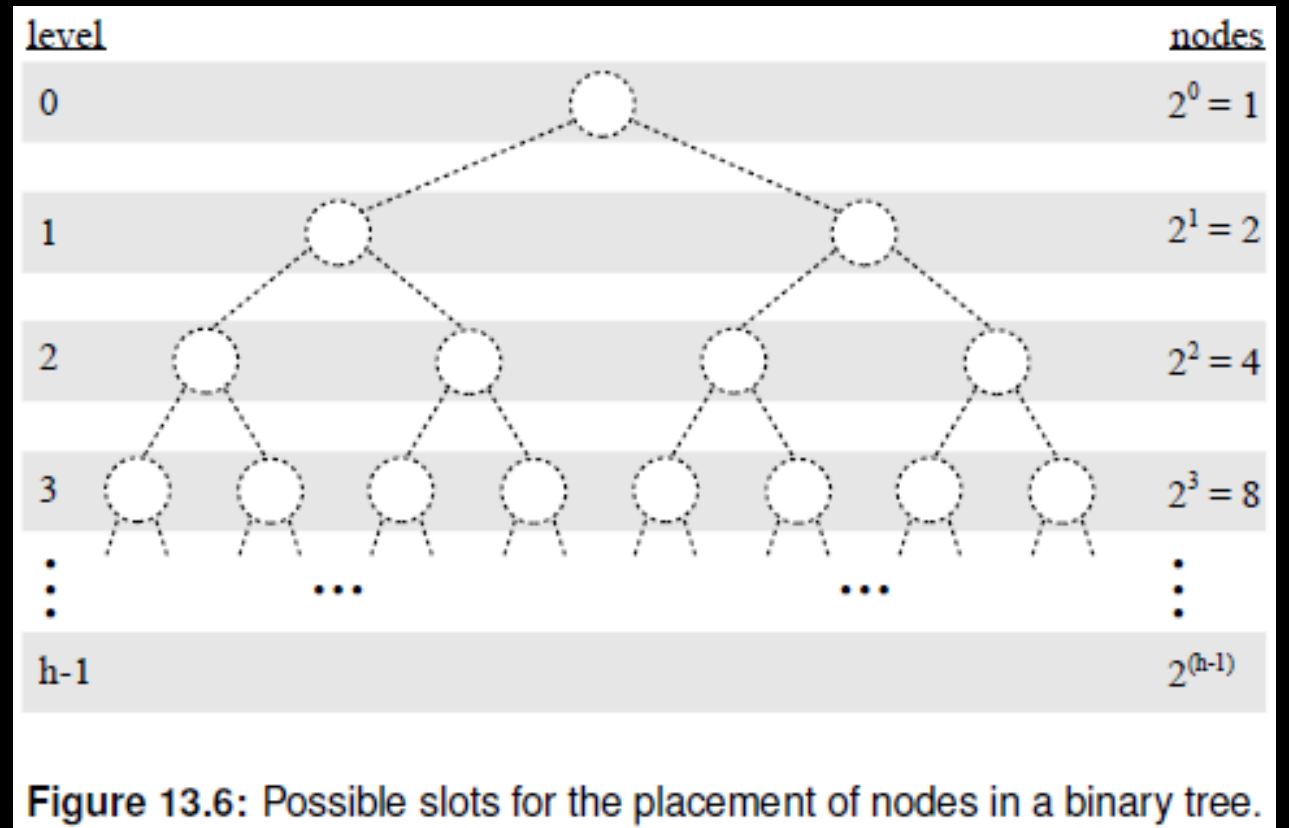
$$n = 2^0 + 2^1 + 2^2 + \dots + 2^{h-1}$$

- This is a finite increasing geometric series, with:

$$a = 2^0, r = 2, \text{terms} = h$$

- In a
max

Level#	Nodes
0	2^0
1	2^1
2	2^2
$\vdots$	$\vdots$
$h - 1$	2^{h-1}



Binary trees

$$n = (2^h - 1)$$

$$2^h = n + 1$$

$$\log_2 2^h = \log_2(n + 1)$$

- When all possible slots contain nodes, and no slot is empty, and the size of the tree is n , then the height of the tree is:

$$h = \log_2(n + 1)$$

- This is the minimum height of the tree.

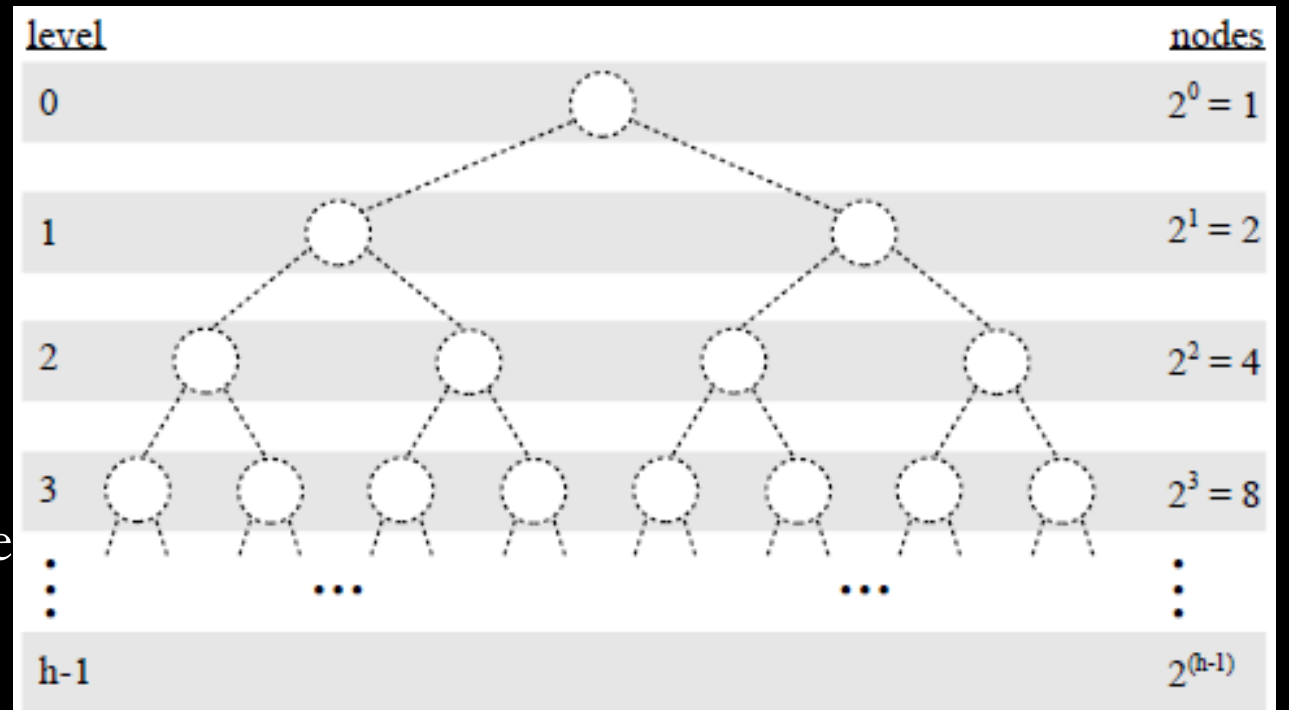


Figure 13.6: Possible slots for the placement of nodes in a binary tree.

Full binary trees

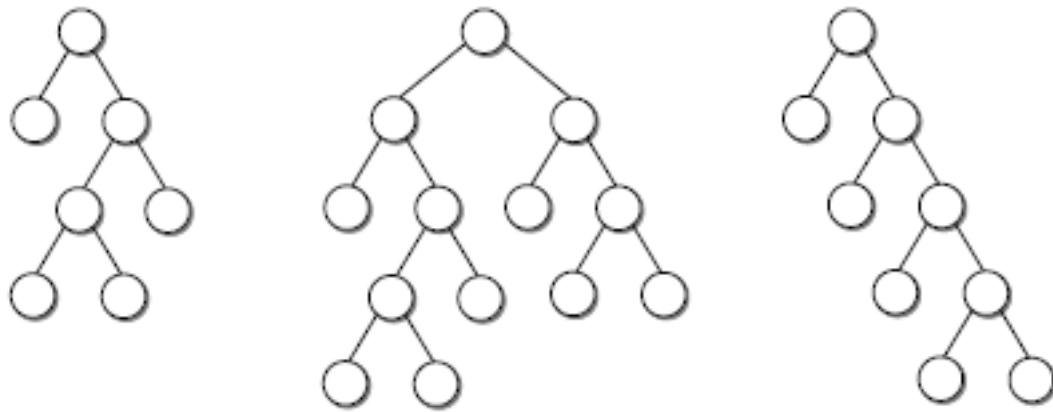


Figure 13.7: Examples of full binary trees.

- A **full binary tree** is a binary tree in which the root and each interior node contains two children.
- It is also called a **strictly binary tree**.
- We use the acronym SBT for the strictly binary tree.



Perfect binary trees

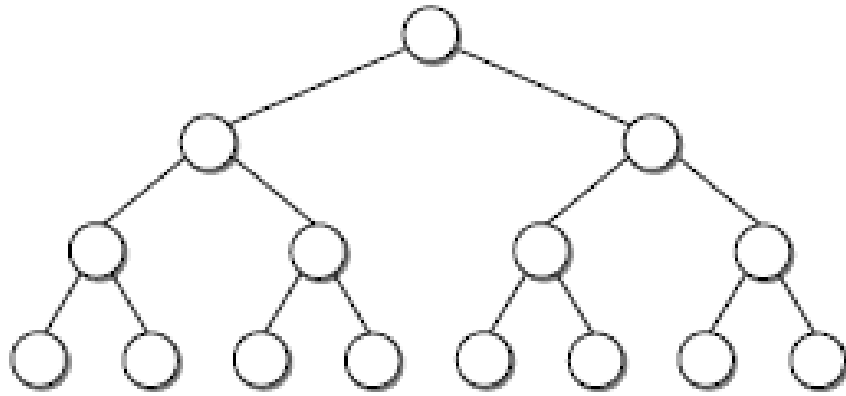


Figure 13.8: A perfect binary tree.

- A **perfect binary tree** is a full binary tree in which all leaf nodes are at the same level.
- The perfect tree has all possible node slots filled from top to bottom with no gaps.
- We use the acronym PBT for the perfect binary tree.



Almost complete binary trees

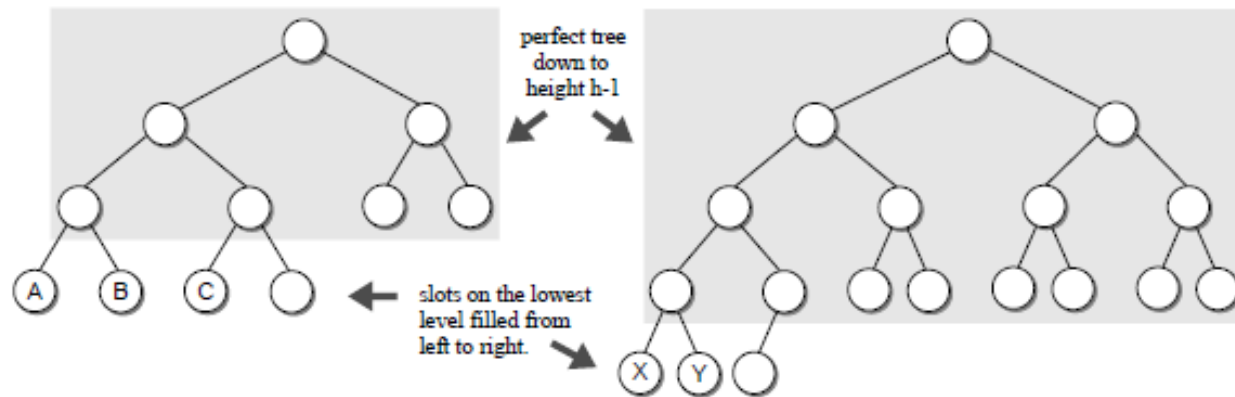
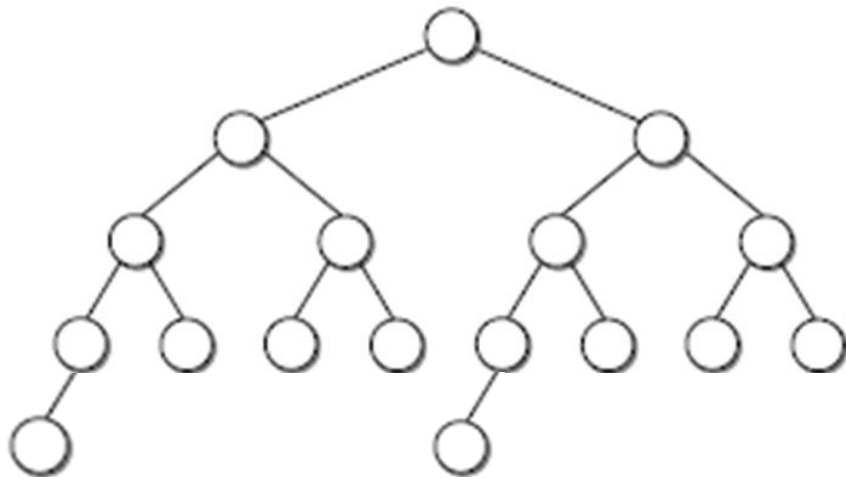


Figure 13.9: Examples of complete binary trees.

- A binary tree of height h is an **almost complete binary tree** if it is a perfect binary tree down to height $h - 1$ and the nodes on the lowest level fill the available slots from left to right leaving no gaps.
- The book calls this tree a **complete binary tree**.
- We use the acronym ACBT for the almost complete binary tree.



Almost complete binary trees

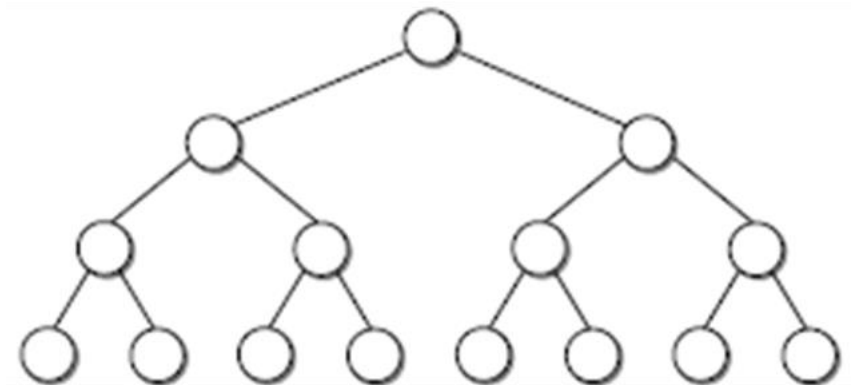


- The left and right subtrees of the root of an ACBT differ in height by 0 or 1.
- Every PBT is also an ACBT.
- If we add another node to a PBT in the leftmost available slot, it ceases to be a PBT, but it still remains an ACBT.
- If a node is added leaving empty slots between the new and existing nodes, the tree ceases to be an ACBT.

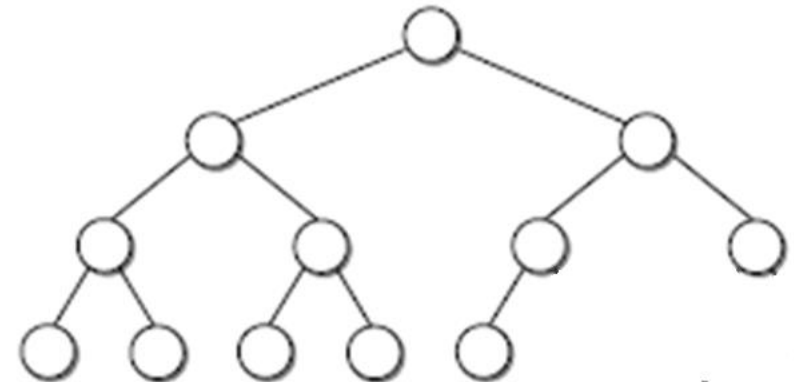


Types of ACBTs

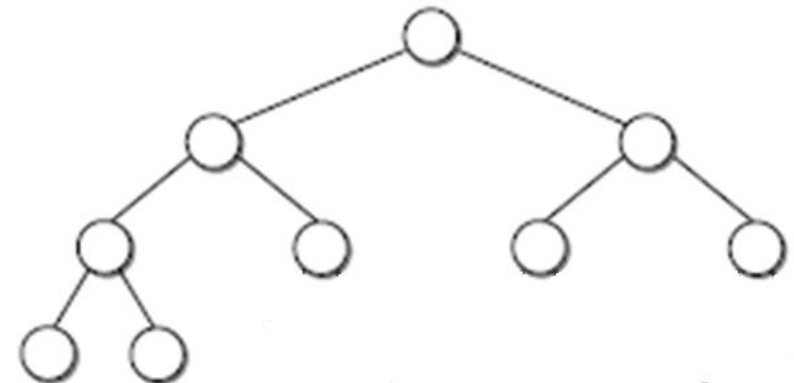
a



b



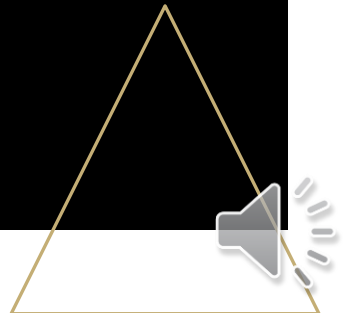
c





When is a binary tree almost complete?

- If the tree is a PBT, then the tree is an ACBT.
- If the difference in height of the two subtrees of the root is 0, the left subtree is a PBT, and the right subtree is an ACBT, then the tree is an ACBT.
- If the difference in height of the two subtrees of the root is 1, the right subtree is a PBT, and the left subtree is an ACBT, then the tree is an ACBT.



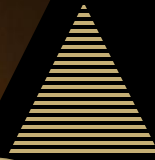
Task

Save your snapshot as `<my_roll_no>_lec19`. (If your roll no. is 500, the file should be named `500_lec19`.)

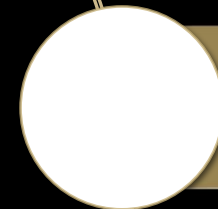
Submit your response in the assignment titled “Lecture 19 tasks” in Google Classroom.

1. Draw all possible ACBTs for 7, 11 and 13 nodes. Do you have more than one trees for each case?
2. Draw an ACBT which is not an SBT.
3. Draw an SBT which is not an ACBT.
4. Draw all possible PBTs of size n ,
 $16 < n < 65$.
5. Consider trees of 0 and 1 nodes. Is the considered tree:
 - a) A PBT?
 - b) An SBT?
 - c) An ACBT?
6. Draw an ACBT and an SBT of 2 nodes.





We were introduced to the tree structure.



We learned about the terminology associated with trees.



We learned about the binary tree.



We learned to classify binary trees.

So, what did we learn today?



Things to do

Read the book!

Note your questions and put them up in the relevant online session.

Submit your answers to the tasks in this lecture.

Email suggestions on content or quality of this lecture at uroojain@neduet.edu.pk

